Finance Data Processing and Access Control Backend

Objective

To evaluate your backend development skills through a practical assignment centered around API design, data modeling, business logic, and access control.

This assignment is intended to assess how you think about backend architecture, structure application logic, handle data correctly, and build reliable systems that are clear, maintainable, and logically organized.

Note: *If you have already built a similar backend project earlier, you may submit that project for evaluation. Please make sure to clearly explain how it matches this assignment and share the repository and, if available, the deployed API or documentation link.*

Key Instructions

- **You are not required to follow a fixed project structure.** You are free to organize the backend in the way you believe is most appropriate.
- **Focus on correctness, clarity, and maintainability.** We are interested in how you design data flow, structure APIs, and write backend logic.
- **Reasonable assumptions are acceptable.** If something is not explicitly defined, you may make sensible assumptions and document them.
- **Clean implementation matters.** A smaller but well designed solution is better than a large but inconsistent one.

Flexibility

You have full freedom to:

- Use any backend language, framework, or library
- Use any database of your choice, or even an in memory store for a simplified implementation
- Define your own schema, service structure, and business logic flow
- Build REST APIs, GraphQL APIs, or an equivalent backend interface
- Use mock authentication and local development setup if needed

---

Scenario

Imagine you are building the backend for a **finance dashboard system** where different users interact with financial records based on their role.

The system should support the storage and management of financial entries, user roles, permissions, and summary level analytics. The goal is to build a backend that is logically structured and able to serve data to a frontend dashboard in a clean and efficient way.

Core Requirements

1. User and Role Management

Provide a way to manage users and their access levels within the system.

Your backend should support:

- Creating and managing users
- Assigning roles to users
- Managing user status such as active or inactive
- Restricting actions based on roles

You may define roles such as:

- **Viewer:** Can only view dashboard data
- **Analyst:** Can view records and access insights
- **Admin:** Can create, update, and manage records and users

The exact role model is up to you, but role based behavior should be clear in your implementation.

2. Financial Records Management

Create backend support for financial data such as transactions or entries.

Each record can include fields such as:

- Amount
- Type such as income or expense
- Category
- Date
- Notes or description

Your backend should support operations such as:

- Creating records
- Viewing records
- Updating records
- Deleting records
- Filtering records based on criteria such as date, category, or type

3. Dashboard Summary APIs

Provide APIs or backend logic that can return summary level data for a dashboard.

Examples include:

- Total income
- Total expenses

- Net balance
- Category wise totals
- Recent activity
- Monthly or weekly trends

The purpose here is to show how you design backend endpoints or service logic for aggregated data, not just basic CRUD operations.

4. Access Control Logic

Implement backend level access control for different roles.

The system should clearly enforce which type of user can perform which action. For example:

- A viewer should not be able to create or modify records
- An analyst may be allowed to read records and access summaries
- An admin may be allowed full management access

You may implement this using middleware, guards, decorators, policy checks, or any equivalent method depending on the framework you choose.

5. Validation and Error Handling

Your backend should demonstrate proper handling of incorrect or incomplete input.

This includes:

- Input validation
- Useful error responses
- Status codes used appropriately
- Protection against invalid operations

The goal is not perfection, but your implementation should show that you understand how a backend should behave in real usage.

6. Data Persistence

Use a persistence approach suitable for your project.

This can be:

- A relational database
- A document database
- SQLite for simplicity
- Any other reasonable option

If you choose a simplified or mock storage approach, mention it clearly in your documentation.

Optional Enhancements

You may include additional improvements such as:

- Authentication using tokens or sessions
- Pagination for record listing
- Search support
- Soft delete functionality
- Rate limiting
- Unit tests or integration tests
- API documentation

These are not mandatory, but thoughtful additions are always appreciated.

Evaluation Criteria

1. Backend Design

How well the application is structured, including routes, services, models, and separation of concerns.

2. Logical Thinking

How clearly business rules, access control, and data processing have been implemented.

3. Functionality

Whether the expected APIs and backend features work correctly and consistently.

4. Code Quality

Readability, maintainability, naming, organization, and general coding practices.

5. Database and Data Modeling

How appropriately data is modeled and managed for the use case.

6. Validation and Reliability

How well the application handles bad input, invalid states, and error conditions.

7. Documentation

Clarity of the README, setup process, API explanation, assumptions made, and any tradeoffs considered.

8. Additional Thoughtfulness

Any extra effort that improves usability, clarity, or system design.

Important Note

This assignment is designed purely for assessment. The purpose is to understand your backend thinking, implementation style, and engineering approach. You are not expected to build a production ready system.

A well reasoned and well structured submission will always be valued more than unnecessary complexity. This assignment is an opportunity to show how you approach backend engineering from both a technical and practical perspective. We are interested not only in whether the system works, but also in how you think while building it.